GOSIM

Maintaining Dora with Agentic Engineering

Keeping code quality high
when most of the code is written by AI

Philipp Oppermann · dora-rs.ai · github.com/dora-rs/dora



Four numbers

1,065

pull requests merged since May

~85%

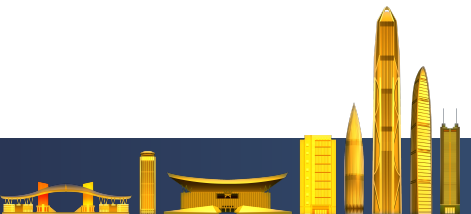
of commits written by an AI agent

~45

merged pull requests per week

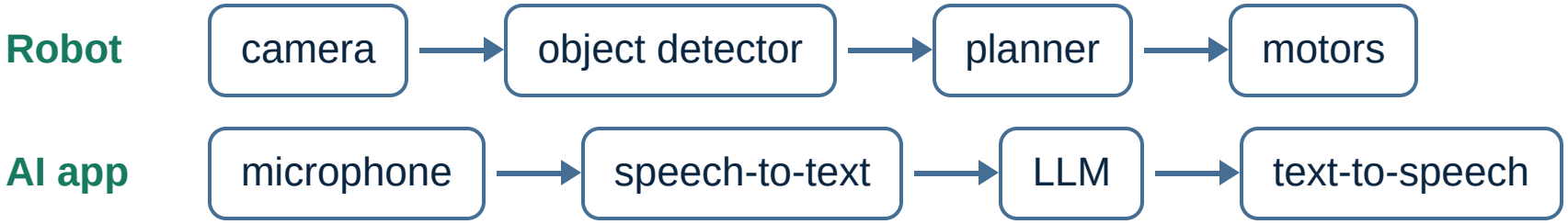
1

maintainer



What Dora is

Dora: a framework for real-time robotics and AI applications



- You describe your application as a **graph of small programs** ("nodes")
- Dora **starts them, connects them, and moves data** between them fast

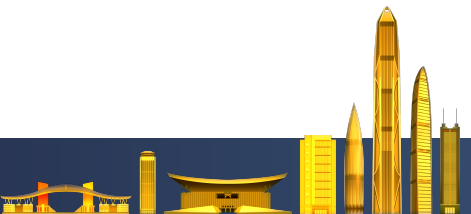
- Nodes can be written in **Rust, Python, C, or C++**
- Runs on **Linux, macOS, Windows**; one machine or many
- Written in **Rust**, **open source** (Apache-2.0), ~4,000 GitHub stars

What makes Dora hard for AI agents

Bugs in Dora tend to be:

- **Non-deterministic**: they show up under load, during shutdown, or on a flaky network
- **Platform-specific**: passes on Linux, fails on Windows
- **Between components**: each part is correct, the combination isn't

With the latest models, AI agents are very capable.
But a **distributed, multi-OS, multi-language system** is still hard for them.



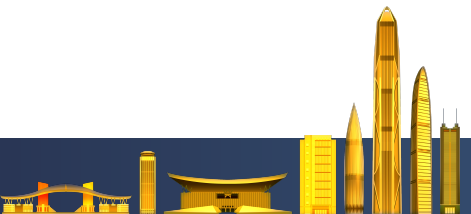
Several layers of checks

Every change goes through five layers of checks:

1. **The language** (Rust): whole classes of bugs don't compile
2. **The architecture**: clear boundaries, one place for each concern
3. **The test suite**: fast checks on every change, deep checks every night
4. **AI reviews**: a second opinion on every pull request
5. **AI routines**: agents that find and fix problems

Plus human review, focused on high-risk changes.

(And at the end: how this all came together for Dora 1.0)



GOSIM

01

The language: Rust



What the Rust compiler checks

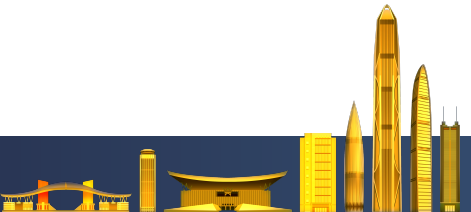
Rust checks a lot before the program ever runs:

- Every value has **exactly one owner** → no use-after-free, no double-free
- Sharing data between threads must be **explicitly safe** → no data races
- "Maybe there's no value" and "this can fail" are **part of the type**

Plus `clippy`: hundreds of extra lint rules, and we treat **every warning as an error**.

If it compiles, whole classes of bugs are ruled out.

Also: Rust's clear error messages are very useful for AI agents.



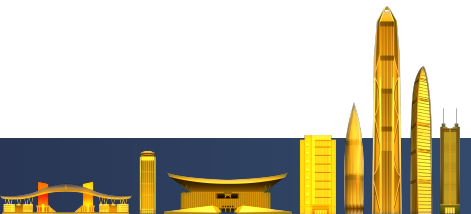
Example: adding a new case


```
enum NodeExit {
    Finished,
    Crashed { code: i32 },
    Restarting,          // ← new case, added by an agent
}

match exit {
    NodeExit::Finished => ...,
    NodeExit::Crashed { code } => ...,
}
// error: non-exhaustive patterns: `NodeExit::Restarting` not covered
```

The compiler lists **every place in the codebase** that has to handle the new case.

(illustrative example, simplified)



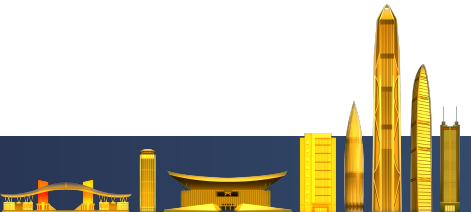
Types that prevent mix-ups


```
struct NodeId(String);
struct OperatorId(String);

fn stop_node(id: NodeId) { ... }

stop_node(operator_id);   // error: expected `NodeId`, found `OperatorId`
```

Both are strings underneath. **The compiler still keeps them apart.**



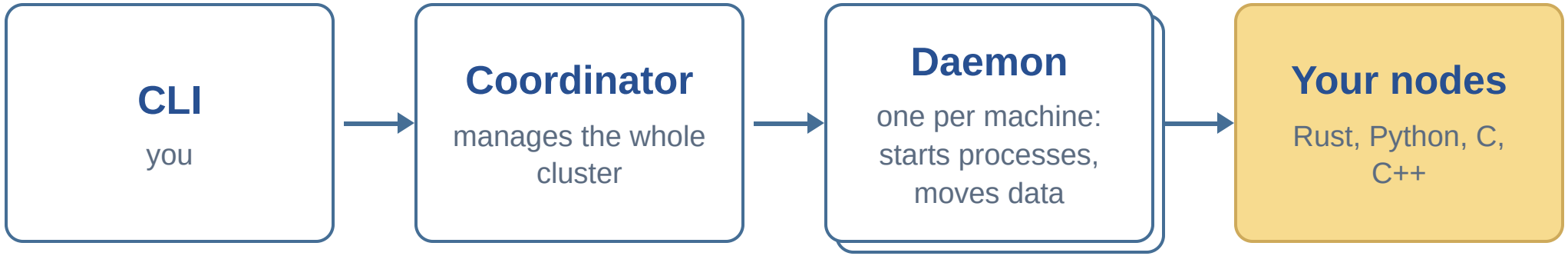
GOSIM

02

The architecture



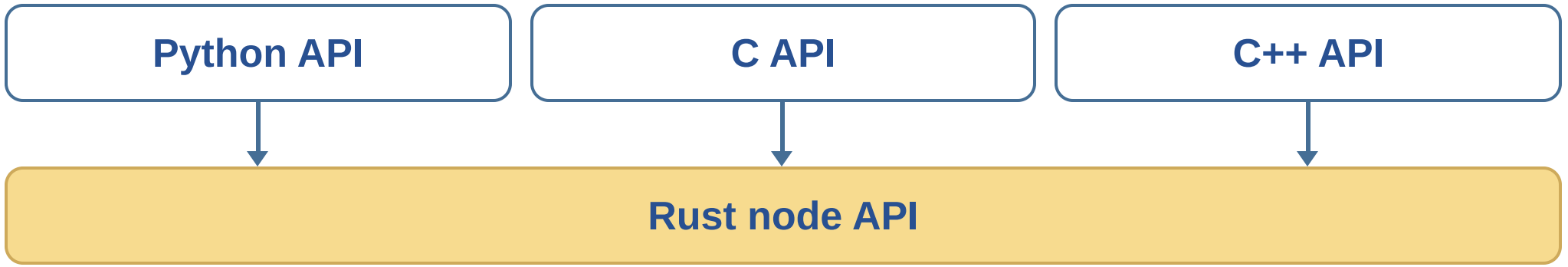
Components with clear interfaces



- Each component has **one job** and a **clear interface** to the others
- The interfaces are defined in one shared crate, as **message types**

It's obvious which changes stay inside one component and which affect others.

Thin wrappers for Python, C and C++



The Python, C and C++ APIs are **thin wrappers** around the Rust API.

- **One implementation** of the node API
- A bug fix in Rust **fixes every language**
- **Nothing to keep in sync** across languages

GOSIM

03

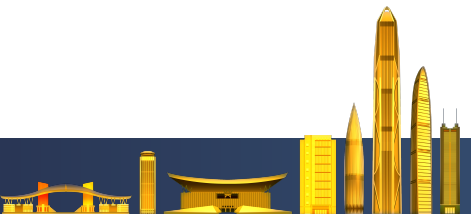
The test suite



About 2,750 tests

Tests	Count	Example
Unit	~2,410	parsing a config value
Integration	~160	the coordinator protocol
End-to-end	~190	run an example dataflow with real processes
Python	~60	the Python node API
Property	~20	the message format, with thousands of random inputs

End-to-end tests run **real processes** over a (local) network, and kill nodes on purpose.



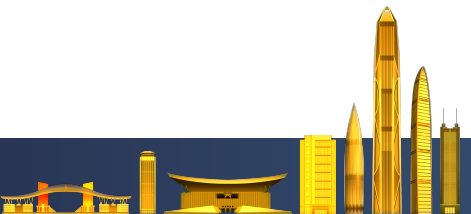
Too many tests for every pull request

- The full test suite takes **~40 minutes** on one OS
- **~45** pull requests a week, on busy days **20 or more**

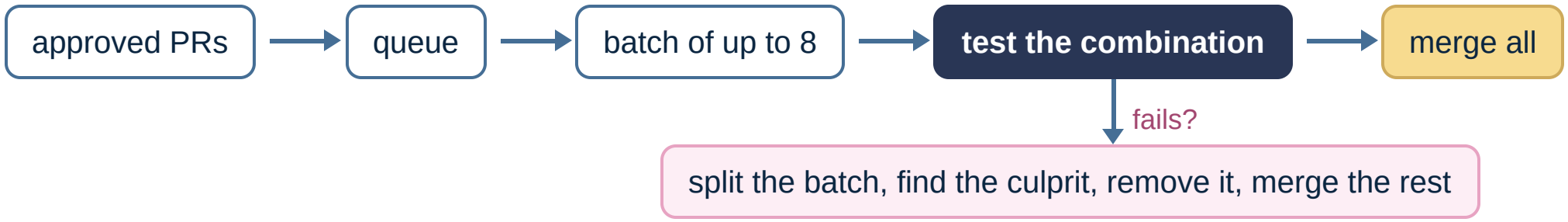
Running the full suite on every pull request **doesn't fit**.

So we split it:

- **On every PR update:** cheap checks, ~10 min
- **Before every merge:** the test suite on Linux, full build on macOS and Windows
- **Every night:** all tests and examples on all 3 OSs: **47 jobs**, the longest over an hour



A merge queue with batching



Pre-merge checks run **once per batch** of up to 8 PRs: **up to 8× less CI time**

Green PRs, `main` still broken

Pull request A: adds a parameter, updates every existing call ✅ tests pass

```
fn register_memory_pool(id: PoolId, metadata: Metadata, owner: String,
                        readers: HashSet<String>)   // ← new
```

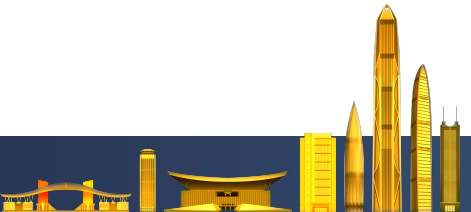
Pull request B: adds a new call, written against the old version ✅ tests pass

```
register_memory_pool(id, metadata, node_id);
```

Merge both:

```
error[E0061]: this function takes 4 arguments but 3 arguments were supplied
```

The merge queue catches this: it tests PRs together, on top of the latest `main`.



Useless tests

The bug: `dora logs --tail 20 --level error` should show the last 20 errors.
It cut to the last 20 lines *first*, then removed everything that wasn't an error → often **nothing** was shown.

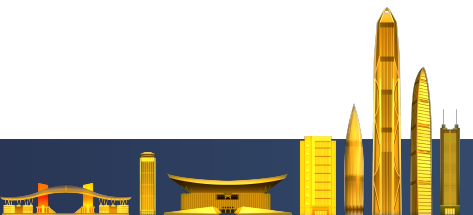
The agent's fix (in all four places that print logs: local, follow, all nodes, single node):

```
// before:
let display = apply_level_filter(apply_tail(messages, tail), level);
// after:
let display = apply_tail(apply_level_filter(messages, level), tail);
```

And the test:

```
#[test]
fn tail_counts_only_level_filtered_lines() {
    let logs = /* 2 errors, then 20 info lines */;
    let shown = apply_tail(apply_level_filter(logs, ERROR), 5);
    assert_eq!(shown, ["err1", "err2"]);
}
```

Revert the fix in all four places → **the test still passes.**



The fixed test

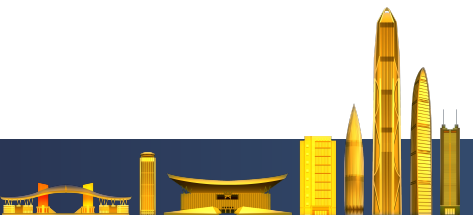

```rust
// Written once
fn filter_and_tail(messages, level, tail) -> Vec<LogMessage> {
    apply_tail(apply_level_filter(messages, level), tail)
}

// Each of the four places that print logs is now one line
let display = filter_and_tail(messages, level, tail);

#[test]
fn tail_counts_only_level_filtered_lines() {
    let logs = /* 2 errors, then 20 info lines */;
    let shown = filter_and_tail(logs, ERROR, 5);
    assert_eq!(shown, ["err1", "err2"]);
}
```

- The test now runs **the same code the users run**: swap the order back → it fails
- Bonus: four copies of the same logic became one

Rule: a regression test must fail on main without the fix.



Mutation testing

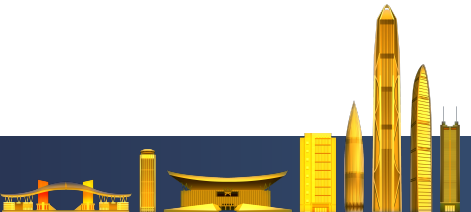
A tool deliberately **introduces small bugs**, then runs the tests:

```
a + b          →   a - b                did a test fail?
if a >= b      →   if a > b             did a test fail?
return list    →   return empty list    did a test fail?
```

If no test fails, that code **isn't really tested**.

Not automated: we run it manually from time to time.

Our tests still miss many of these small bugs.



GOSIM

04

AI reviews



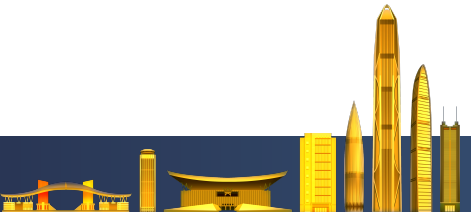
Every pull request gets an AI review

Every 2 hours, a scheduled routine reviews every open pull request:

- currently with **Claude Opus 5.5**
- a **fresh review** every time: AI isn't deterministic, so repeats **find new things**
- every comment is clearly labeled as machine-generated
- after new commits: which findings are **resolved**, which are **still open**

How well it works (961 PRs since June):

- findings are **mostly right**: ~4% clearly wrong, ~15% overstated
- "no issues found" is weak: in **~1 in 4** PRs, something was found later
- repeats pay off: **42%** of findings came from re-reviewing an unchanged commit



The review prompt

Review all open PRs. Be diligent and take your time.

Don't trust the PR description or existing review comments.
The code and the diff is the only source of truth.

Validate that the change is really needed and that there is
no simpler way to achieve the goal.

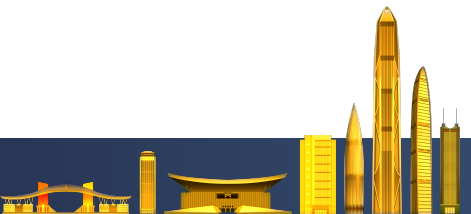
Make sure that added tests are really testing something useful.
Regression tests should fail on main (i.e. when run without the fixes).

Only report the most important issues, I don't care about nits.

You're not allowed to "approve" a PR. Report your findings like an outside contributor.

Don't post a comment at all when there are no new findings
since the last review comment.

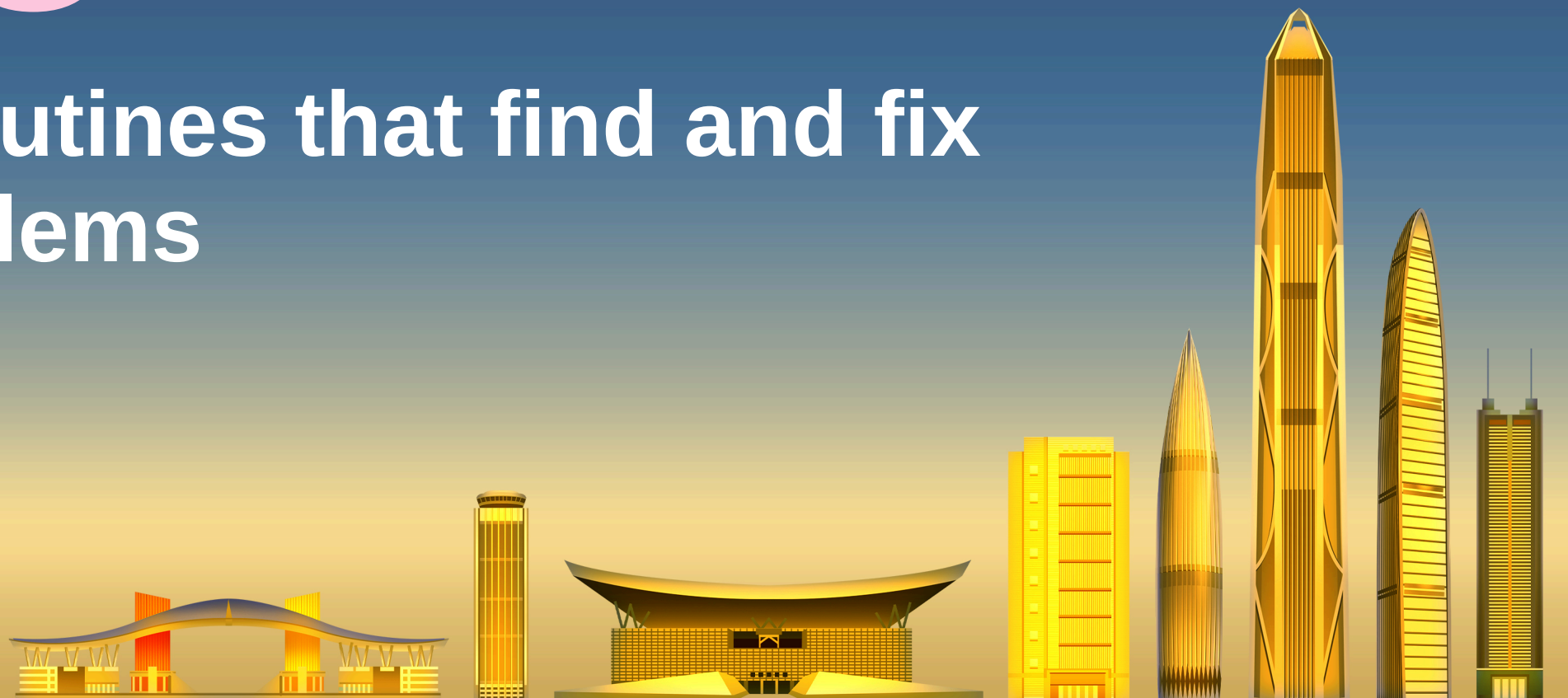
(the actual prompt, shortened)



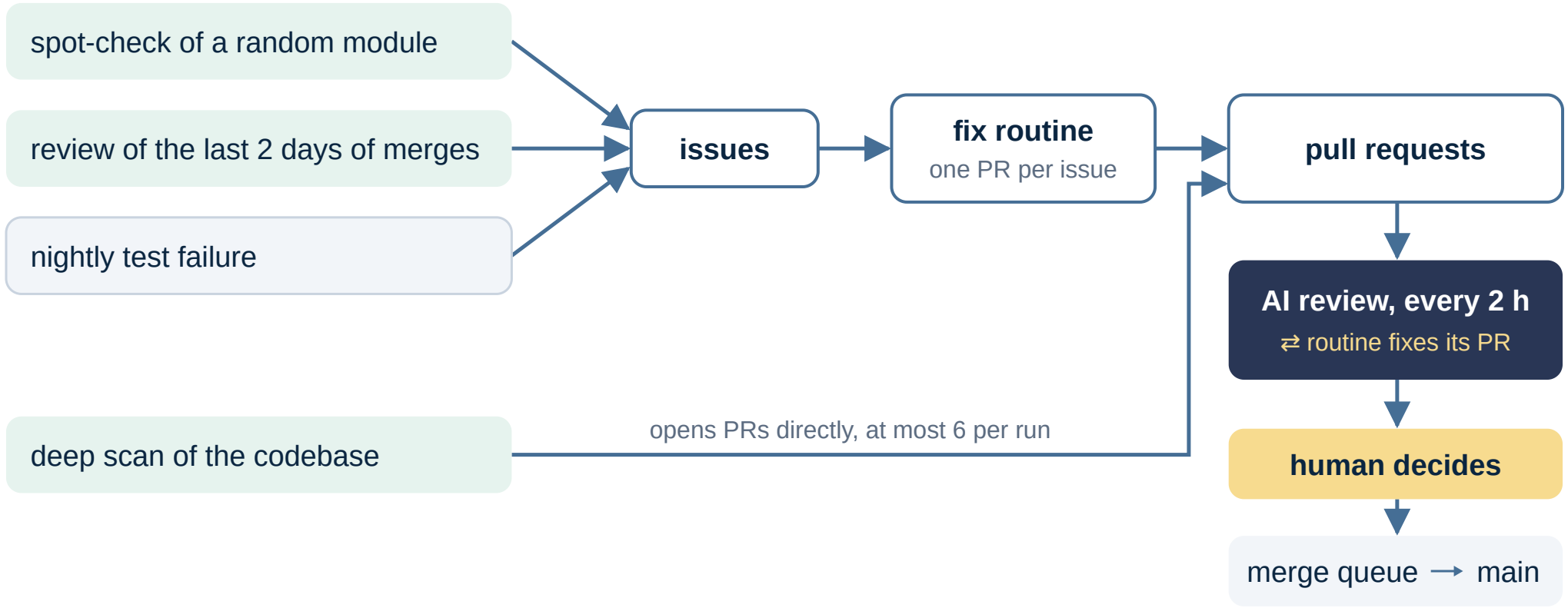
GOSIM

05

AI routines that find and fix problems



Agents that find and fix problems



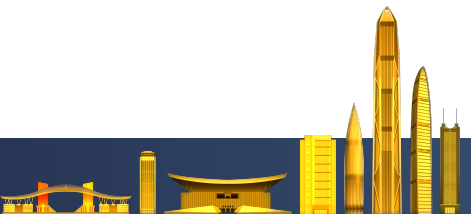
Since May: **412** issues opened, **376** closed. **36** open today.

Spot-check of a random module

"Randomly select a feature or component from the codebase and review it for potential bugs or correctness issues."

Bugs it found:

- `dora logs --node cam` **also showed the logs** of node `cam.left` (it matched every log file starting with `log_cam`)
- Two nodes with the same name: one was **silently dropped** instead of reporting an error



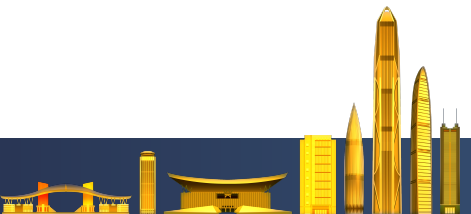
Review what merged in the last two days

"Review recent (last 2 days) commits and pull requests … for potential bugs or regressions. … also consider how the PRs/commit interact or conflict with each other."

Red flags: skipped tests, overly broad error handling, **incomplete refactors**

Example:

- Older feature: `dora replay` puts every input into "never drop messages" mode
- New PR: that mode now routes data through the daemon, **without zero-copy**
- Each change makes sense; together, every replay **silently lost zero-copy**
- Issue filed the next night



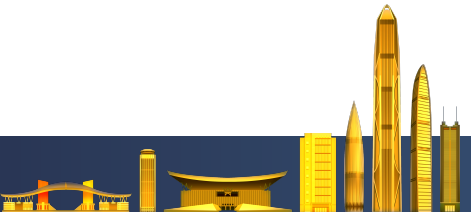
The deep scan

Looks for four things:

- **Correctness**: logic errors, edge cases, crashes, security problems
- **Performance**: low-hanging fruit, hot paths first
- **Documentation**: gaps in the public API, as compile-checked examples
- **Simplification**: dead code, duplication, needless complexity

Each run:

- opens up to **6** pull requests
- each explains the problem, the fix and how it was tested
- lists the findings it rejected



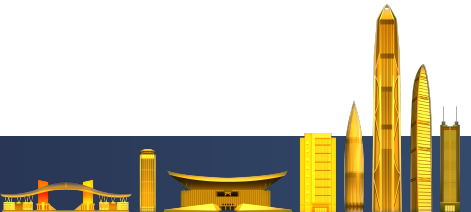
What the deep scan finds

~150 pull requests since late June, ~140 merged:

Bug fixes	Input hardening	Docs	Performance	Simplification
~70	~35	~25	~13	~12

Bugs it fixed:

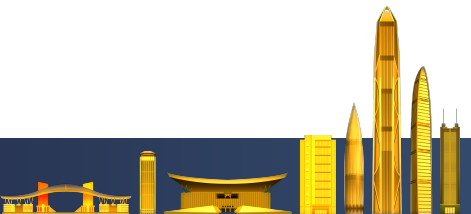
- **Serial ports**: device paths with a colon couldn't be opened
- **Secret token file**: a new secret could end up in a file **everyone could read**
- **Restart delay**: a huge (but allowed) delay crashed the restart logic
- **Log files**: machines `10.0.0.1` and `10.0.0.2` wrote into the same log file



Routines check each other

Sep 24, 09:49	deep scan	PR merged: two copies of a size parser → one the shared one used a type that can hold only 4 GB on 32-bit systems
Sep 25, 02:10	merge review	files an issue: "8 GB log limit is now rejected on 32-bit targets"
Sep 25, 03:28	fix routine	opens a pull request
Sep 25, 14:47	human	merged

The bug was fixed about 29 hours after it was merged.



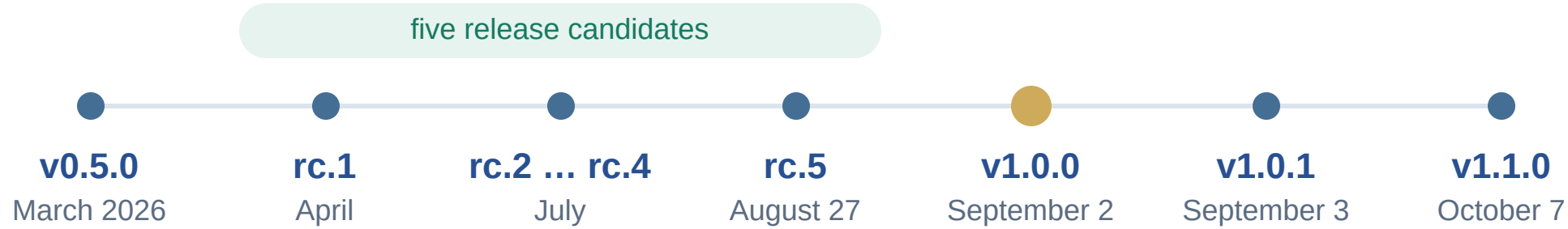
GOSIM

06

The road to Dora 1.0



1.0 is a stability guarantee



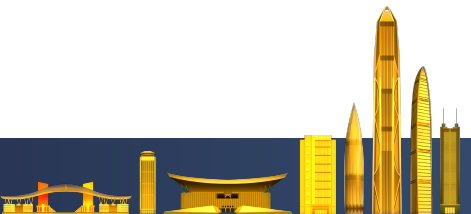
1.0 = "code and configs you write today keep working for all of 1.x."

What we promise, and how we check it

Tier	What	Rule
Stable	Node APIs (Rust, Python, C, C++), the message protocol, the `dora` command and the config format	frozen until 2.0
Unstable	ROS 2 bridge, operators, node hub, MAVLink bridge, GPU tensor pool	may change in a minor release; opt-in
Internal	daemon, coordinator, core libraries	no promise; published only because Cargo needs it

Every stable surface is checked against the last release:

`cargo-semver-checks` for Rust · `dora` CLI snapshot · config schema diff

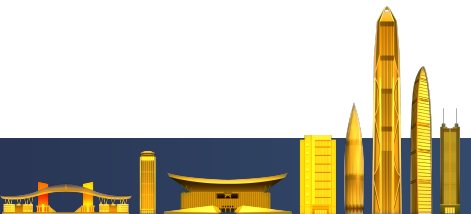


Breaking changes before 1.0

Before 1.0, we broke things on purpose:

- **Replaced the message encoding**: the old library was unmaintained, with no spec
- **Made message types extensible**: they can grow later without a 2.0
- **Took Apache Arrow's version out of the promise**: we can upgrade it in 1.x

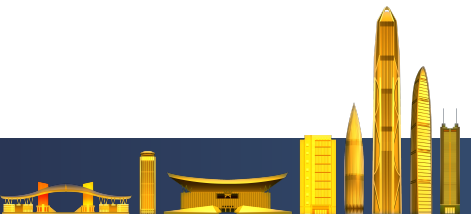
Every break gives a clear error message instead of silently wrong data.



How we decided 1.0 was ready

- Shipped after **11** nightly CI runs in a row with all jobs green
- Five release candidates
- No open critical bugs
- Prepared for backwards and forwards **compatibility**, e.g. version checks on connect
- Benchmark: **7–39% lower latency** than 0.5 at every message size
- Migration guide for users of 0.x

1.0 is about stability guarantees, not about being finished.



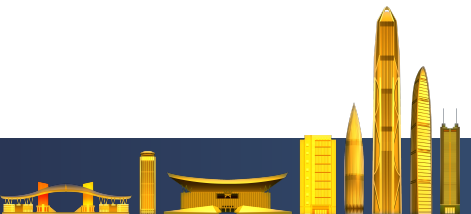
And then 1.0.1, the next day

Dora 1.0.0: **every check green**, and the prebuilt binaries worked.
But building from source with `cargo install dora-cli` still **failed**.

The reason:

- One package read a file from a **neighboring** folder
- In the repository, that folder exists → **all tests pass**
- In the published package, only its own files are included → **build fails**

Fixed in 1.0.1, plus a **new check**: a package may only use its own files.



GOSIM

07

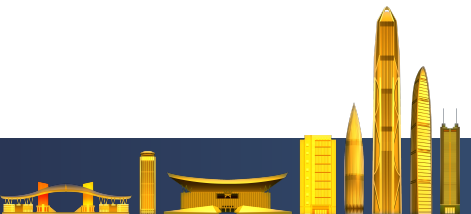
Closing



What the human does

- Decides **what** to build, and in which order
- Writes and reviews the **plans for big features**
- Reads **high-risk changes carefully**; skims low-risk ones
- Checks the **approach**: hacky, too complicated, or only fixing the symptom?
- Decides when **not** to fix something

Every merge is a human decision.



Takeaways

1. **Pick a strict programming language** and **write modular code.**
2. **Build an extensive, high-quality test suite**, and check that your tests can fail.
3. **Use AI to review and to find problems.**
4. **Check stability promises automatically**, so an accidental break fails the build.
5. **A human is still needed.** Review large or high-risk changes yourself.

Thank you!

Learn more about Dora at **dora-rs.ai** and **github.com/dora-rs/dora**

